FRED 2.0

User Quickstart Manual

FRED-ROD and FRED-OX — a step-by-step guide through the example scripts

FRED Development Team
Paul Scherrer Institut / Advanced Nuclear System group

July 17, 2026

Contents

1 Introduction

FRED 2.0 is a platform for developing fuel-performance codes. Core physics — heat conduction and thermo-elastic stress-strain, including pellet-cladding gap closing/reopening — lives in a shared platform layer. Applications are built on top of that layer:

FRED-ROD solves thermal conduction and stress-strain only. No irradiation behaviour is modelled; users supply their own material correlations (including, if desired, their own base-irradiation effects). This decoupling is deliberate: it keeps FRED-ROD suitable for controlled experimental/verification workflows.

FRED-OX uses the same equations as FRED-ROD and adds oxide-fuel-specific physics on top: empirical base-irradiation correlations for fuel/cladding properties, and fission-gas production, which feeds back into gas pressure and gap conductance.

FRED-M-Na uses the same equations as FRED-ROD and adds metallic-fuel-specific physics (U-Pu-Zr in sodium), including many correlations ported from SAS4A. See `examples/fred_m_na/` and `doc/physics_fred_m_na.md`; Section 25 of this manual gives a short pointer to its examples.

This manual covers FRED-ROD (Part I) and FRED-OX (Part II) in depth, walking through every example script under `examples/fred_rod/` and `examples/fred_ox/` in a recommended learning order, from the simplest case to the most complex. Each part opens with its own "recommended example order" list.

Prerequisites

All examples assume the relevant FRED Python module has been compiled and placed in `fred_platform/build/`. Add that directory to your Python path:

```
# from the fred_platform root
make pymodule              # builds build/fred_rod.*.so, fred_ox.*.so
   , ...
export PYTHONPATH=$PWD/build:$PYTHONPATH
```

Each script also inserts the build path automatically with `sys.path.insert(0, ...)`, so you can run them directly from their own directory without setting `PYTHONPATH`.

1.1 Common workflow

Every FRED-ROD or FRED-OX script follows the same four-step pattern:

1. **Geometry** — set the node counts, radii, axial slice heights, roughness, and gas-plenum volume; call `build()`.
2. **Materials** — choose or define fuel, cladding, and gap material objects.
3. **Solver setup** — create a `FredRodSolver` (ROD) or `FredOxSolver` (OX), set physics toggles, boundary conditions, tolerances.
4. **Run and post-process** — call `run(tend, dtout)`; retrieve results with the accessor methods.

1.2 Unit conventions

Quantity	Unit	Notes
Temperature	K	Always Kelvin; subtract 273.15 for °C
Time	s	Seconds
Length / radius	m	Metres
Power density	W/m^3	Volumetric heat generation
Pressure / stress	MPa	Megapascals

2 Time Control Features

Every FRED application (`FredRodSolver`, `FredOxSolver`, `FredMNaSolver`) shares the same time-control machinery around `run(tend, dtout)`: the solver takes its own internal time steps (adaptive IDA steps for FRED-ROD/FRED-OX, fixed backward-Euler steps for FRED-M-Na) and produces *output events* every `dtout` seconds. Two persistence mechanisms hook into those output events.

2.1 Variable output schedule (vector `dtout`)

`dtout` may be a scalar (constant output interval, as in every example above) *or* a sequence of successive intervals:

```
# very fine early, coarse later (sums to >= tend)
solver.run(tend=200.0,
           dtout=[0.1]*10 + [0.5]*8 + [2.0]*10 + [25.0]*7)
```

Small early intervals do more than refine the output sampling: for FRED-ROD/FRED-OX each output time is an IDA stop time, so a fine early schedule also bounds the solver's internal step through the stiffest part of a run (power step, contact onset) and can help it through numerical instabilities; for FRED-M-Na the internal backward-Euler step follows the schedule whenever no explicit `set_step_size` is given. If the list ends before `tend` the last entry repeats, and the Python wrappers require `np.sum(dtout) >= tend` so an under-covering schedule is rejected up front. Checkpoints and snapshots (below) fire at the scheduled output events exactly as with a scalar `dtout`. See `examples/fred_rod/dtout_example/` for a scalar/vector pair of scripts that validate the produced output times against the request.

2.2 Checkpoint files — crash recovery and debugging

A checkpoint is a single, overwriting binary file (`<prefix>_checkpoint.chk`) intended for fault recovery. It is disabled by default; enable it by giving the solver a file prefix before `run()`:

```
solver.set_checkpoint_prefix("ckpt/myrun")    # -> ckpt/
    myrun_checkpoint.chk
solver.run(tend, dtout)
```

Once enabled, the checkpoint is rewritten **at every `dtout` output event** — not at every internal step — so after a crash (power loss, cluster time limit, a solver failure you are investigating) at most one output interval of progress is lost. To resume:

```
solver = make_solver()                          # same geometry/
    materials/BCs
solver.load_checkpoint("ckpt/myrun_checkpoint.chk")
print(solver.restart_time())                    # saved physical time [s
    ]
solver.run(tend, dtout)                         # continues from
    restart_time()
```

Simulation time is *preserved*: `restart_time()` returns the time at which the checkpoint was written, and the run continues from there. Restarted trajectories match an uninterrupted reference run to within 10^{-6} relative error (see the comparison scripts referenced below).

Checkpoints as a debug tool

Beyond crash recovery, checkpoints are useful when hunting a failure that occurs late in a long run: keep checkpointing enabled, let the run fail, then reload the last checkpoint and re-run only the final interval — with more output (`all_steps=True`), higher verbosity, or a modified setting — instead of repeating hours of preceding history. Since the file is overwritten in place, choose a smaller `dtout` if you want the recovery point to sit closer to the failure.

2.3 Snapshot files — staged simulations (base irradiation → transient)

A snapshot is a permanent restart-state file: the full physical state (temperatures, strains, gap state, burnup/fission-gas state in FRED-OX and FRED-M-Na) is saved, but on reload the simulation clock is *reset to zero*. This is the mechanism for multi-stage studies where a long steady-state base-irradiation run pre-conditions the fuel, and one or more fast transients are then started from that state with fresh time coordinates:

```
# Stage 1: base irradiation, saves numbered frames
solver.set_snapshot_prefix("snap/esfr_pin_base",
                           snapshot_times=[t_mid])  # optional extra
                              frames
solver.run(t_irr, dtout)       # final frame is always saved
   automatically
# -> snap/esfr_pin_base_frame1.snapshot, ..._frame2.snapshot

# Stage 2: transient from the irradiated state
solver2 = make_transient_solver()
solver2.load_snapshot("snap/esfr_pin_base_frame1.snapshot")
print(solver2.restart_time())  # 0.0 -- time reset, state preserved
solver2.run(t_transient, dtout_transient)
```

Snapshots are written at the user-supplied `snapshot_times` (matched to the nearest output event) and always at the end of the run; unlike the checkpoint they are numbered (`<prefix>_frame1.snapshot`, `_frame2`, ...) and never overwrite each other, so you can build a library of pre-conditioned states (e.g. several burnup levels) and launch transients from each.

Worked examples

The complete base-irradiation→transient workflow is `examples/fred_ox/ESFR_pin/` (Section 22): the `base_irradiation` script saves an end-of-irradiation snapshot, and the `ULOF_transient` / `UTOP_transient` scripts load it to run 43 s and 1000 s transients from the 600-day irradiated state. The mechanics of both features are demonstrated step by step in `examples/fred_rod/restart_snapshot/` (Section 10) and their FRED-OX counterparts (Sections 20 and 21).

Part I
FRED-ROD

FRED-ROD is the thermo-mechanical application of the FRED 2.0 platform. It solves the coupled heat-conduction and thermo-elastic stress-strain equations for a cylindrical fuel rod using

the SUNDIALS IDA differential-algebraic solver. No irradiation behaviour is modelled; users add their own material models (including, if desired, their own base-irradiation correlations).

3 Recommended Example Order

The example scripts under `examples/fred_rod/` are best worked through in the following order, progressing from the simplest case to the most complex:

1. `heat_conduction_only` — heat equation alone; no mechanics.
2. `stress_strain_only` — elastic mechanics alone; temperatures fixed.
3. `heat_conduction_stress_strain` — coupled thermo-elastic run.
4. `gapclose_reopen` — gap close and reopen under power cycling.
5. `hot_start_demo` — skipping the transient heat-up with a pseudo-time hot start, and when *not* to use it.
6. `hdf5_output` — writing and reading results in HDF5 format.
7. `restart_snapshot` — fault-recovery checkpoints and physical-state snapshots.
8. `dtout_example` — constant vs. variable output-interval schedules (scalar and vector `dtout`).
9. `python_materials` — defining custom material correlations in Python.
10. `advanced_prototyping` — the Python→C++ prototyping loop for gap materials.

Why this order?

Examples 1–3 build up the physics one piece at a time (thermal only, mechanical only, then coupled) using the same dummy-material rod, so you can isolate solver behaviour before adding complexity. Example 4 introduces the gap-contact event logic that most real rods exercise. Example 5 (hot start) is easiest to understand *after* Example 3, since it directly replaces the manual power-ramp workaround used there. Examples 6–8 are about persistence, I/O, and time control, independent of the underlying physics, so they can be read once you are comfortable with any of the earlier runs. Examples 9–10 are for users who want to extend FRED-ROD with new material correlations, and assume familiarity with Examples 1–3.

4 Example 1: Heat Conduction Only

File: `examples/fred_rod/heat_conduction_only/run.py`

This is the simplest possible run: heat conduction is enabled and the stress-strain solver is disabled. All mechanical state variables are pinned to zero; the only degrees of freedom are the nodal temperatures.

Scenario:

- Solid dummy fuel pellet and dummy cladding (constant properties).
- Constant volumetric power $q''' = 10^8$ W/m^3 applied at $t = 0$.
- Constant coolant temperature $T_{\text{cool}} = 700$ K.
- Simulated for $t_{\text{end}} = 10$ s (about 30 thermal time constants).

Geometry setup.

```
g = fred.FuelRodGeometry()
g.nf   = 3          # 3 radial fuel nodes
g.nc   = 2          # 2 radial cladding nodes
g.nz   = 1          # 1 axial layer
g.rfi0 = 0.0        # solid pellet (no central void)
g.rfo0 = 4.2e-3     # outer fuel radius = 4.2 mm
g.rci0 = 4.3e-3     # inner cladding radius (100 µm as-fabricated gap)
g.rco0 = 4.9e-3     # outer cladding radius = 4.9 mm
g.dz0  = [0.10]     # one 10 cm axial slice
g.vgp  = 1.0e-6     # 1 cm^3 gas plenum
g.ruff = 5.0e-6     # 5 µm fuel surface roughness
g.rufc = 5.0e-6     # 5 µm cladding inner roughness
g.build()
```

The `nf` and `nc` fields set the number of radial nodes in the fuel and cladding respectively. With `nf=3, nc=2` there are 5 unknown temperatures per axial layer. The as-fabricated gap width is $r_{\mathrm{ci0}} - r_{\mathrm{fo0}} = 100$ µm.

Physics toggles and boundary conditions.

```
fuel = fred.DummyFuelPellet()     # k=10 W/mK, rho=10000 kg/m3, cp=100
    J/kgK
clad = fred.DummyCladding()
gap  = fred.DummyGapMaterial()    # h_gap(T) = 500 + 2.5*T  [W/(m2·K)]

solver = fred.FredRodSolver(g, fuel, clad, gap)
solver.set_enable_heat_conduction(True)    # thermal ODE active
solver.set_enable_stress_strain(False)     # mechanics frozen at zero

T0 = 700.0    # K
solver.set_initial_temperature(T0)
solver.set_coolant_temperature([0.0, 1e6], [T0, T0])     # constant at
    T0
solver.set_power_density_history([0.0, 1e6], [1.0e8, 1.0e8])  # step
    on at t=0

solver.set_coolant_pressure(5.0)      # MPa (required but not used here
    )
solver.set_initial_gas_pressure(0.1) # MPa
solver.set_tolerances(1e-6, 1e-8)
```

`set_coolant_temperature` prescribes the outer cladding surface temperature (Dirichlet boundary condition). `set_power_density_history` provides the volumetric heat source; values are linearly interpolated between the supplied time points.

Analytical check. For a solid cylinder the steady-state fuel centreline temperature is:

$$T_{\mathrm{cl}} = T_{\mathrm{cool}} + q''' R_f^2/(4k_f) + q''' R_f^2 \ln(r_{\mathrm{ci}}/r_{\mathrm{fo}})/k_{\text{gap-eff}} + \dots \tag{1}$$

With the constant-property dummy materials the expected centreline rise above 700 K is approximately $\Delta T \approx 137$ K, giving $T_{\mathrm{cl}} \approx 837$ K. The example prints a comparison table against the legacy FRED Fortran code.

Post-processing.

```
times  = solver.time_points()          # [n_steps] s
T_arr  = solver.temperatures()         # shape (n_steps, nz*(nf+nc))
T_peak = solver.peak_fuel_temperature()  # [n_steps] K
```

```
# Temperature at the innermost fuel node at the final time step:
T_centre_final = T_arr[-1, 0]

# Temperature at the outer cladding surface:
T_clad_outer_final = T_arr[-1, nf + nc - 1]
```

The `temperatures()` array has shape `(n_steps, nz*(nf+nc))`. For `nz=1` it collapses to `(n_steps, nf+nc)`. Nodes are ordered fuel inner→outer, then cladding inner→outer.

Calling `set_enable_stress_strain(False)` reduces the DAE to a pure ODE system (all thermal equations are differential). The solver runs faster and the IDA initialisation is simpler. Use this mode whenever mechanics are not needed.

5 Example 2: Stress-Strain Only

File: `examples/fred_rod/stress_strain_only/run.py`

Heat conduction is disabled; all temperatures are frozen at the initial value `T0 = 293.15 K` (the reference temperature for thermal expansion, so all thermal strains are zero). Only the thermo-elastic mechanics are active.

Scenario:

- Coolant pressure ramp from 5 MPa to 0.1 MPa over 100 s.
- Internal gas pressure constant at 0.1 MPa.
- As the external compression decreases, the cladding expands outward.

Physics and boundary conditions.

```
solver.set_enable_heat_conduction(False)   # temperatures frozen at T0
solver.set_enable_stress_strain(True)

T0 = 293.15   # K ---reference temperature (zero thermal expansion)
solver.set_initial_temperature(T0)

# Coolant pressure ramps from 5 MPa down to 0.1 MPa over 100 s
solver.set_coolant_pressure_history([0.0, 100.0], [5.0, 0.1])
solver.set_initial_gas_pressure(0.1)   # MPa
```

When heat conduction is disabled, the thermal ODE is replaced by an algebraic constraint that pins every node temperature to `T0`. The stress-strain system is then quasi-static: at each output step FRED-ROD solves the mechanical equilibrium equations using the current (frozen) temperature distribution and the instantaneous pressure boundary conditions.

Expected steady-state stress. The thin-wall approximation for the cladding hoop stress under external pressure p_o and internal pressure p_i is:

$$\sigma_\theta \approx \frac{p_i r_i - p_o r_o}{r_o - r_i}. \tag{2}$$

At $t = 0$: $\sigma_\theta \approx (0.1 \times 4.3 - 5.0 \times 4.9)/0.6 \approx -40$ MPa (compressive under external pressure). At $t = 100$ s: $\sigma_\theta \approx -1$ MPa.

Results retrieval.

```
times      = solver.time_points()
sigh_outer = solver.clad_outer_hoop_stress()  # [n_steps] MPa
rco        = solver.clad_outer_radius()        # [n_steps] m

delta_rco_um = (rco[-1] - g.rco0) * 1e6   # outer radius change in μm
```

Key learning point

Use `stress_strain_only` to verify the mechanical solver in isolation. With temperatures fixed at T_{ref} all thermal expansion strains vanish, leaving pure elastic deformation under prescribed pressure. This is the easiest state to compare with the Lamé analytical solution.

6 Example 3: Heat Conduction + Stress-Strain

File: `examples/fred_rod/heat_conduction_stress_strain/run.py`

Both physics blocks are active simultaneously. The heat equation generates a temperature distribution which drives thermal expansion, while the mechanical equilibrium is solved at each output step using the latest temperatures. This is the *operator-split* coupling used in FRED-ROD: IDA advances the thermal ODE, and the mechanics are updated after each output time.

Scenario:

- Slow power ramp $0 \to 10^8$ W/m^3 over 50 s, then held constant.
- Coolant at 700 K, external pressure 5 MPa.
- Simulated to $t = 200$ s (well past steady state).

Why a slow power ramp?

```
# Ramp 0→1e8 W/m3 over 50 s, then hold
solver.set_power_density_history(
    [0.0, 50.0, 100.0, 1e6],
    [0.0, 1.0e8, 1.0e8, 1.0e8])
```

When both heat conduction and stress-strain are active, the IDA initial condition calculation (`IDACalcIC`) must find consistent initial derivatives for the coupled system. A step change in power at $t = 0$ introduces a large discontinuity that can cause `IDACalcIC` to fail. Ramping the power over ≥ 50 s gives the solver time to build a smooth consistent initial state.

Enabling both physics.

```
solver.set_enable_heat_conduction(True)
solver.set_enable_stress_strain(True)

T0 = 700.0    # K
solver.set_initial_temperature(T0)
solver.set_coolant_temperature([0.0, 1e6], [T0, T0])
solver.set_coolant_pressure(5.0)         # MPa
solver.set_initial_gas_pressure(0.1)     # MPa
```

Accessing combined results.

```
T_arr = solver.temperatures()      # (n_steps, nf+nc)  K
sigh  = solver.clad_outer_hoop_stress()  # (n_steps,)  MPa
rco   = solver.clad_outer_radius()       # (n_steps,)  m

# Centreline and cladding inner-surface temperatures at each step:
T_ctr = T_arr[:, 0]           # innermost fuel node
T_oci = T_arr[:, g.nf]        # innermost cladding node (= clad inner
    surface)
```

Expected steady-state behaviour.

- Fuel centreline: ≈ 837 K (44 K above outer fuel surface).
- Cladding outer radius grows by ≈ 17–19 µm (thermal expansion from $T_0 \approx 293$ K reference to the cladding operating temperature).
- Hoop stress is compressive (≈ -28 MPa) because the external 5 MPa coolant pressure dominates over the thermal expansion effect.

Power-ramp requirement

Always use a slow power ramp (*not* a step at $t = 0$) when both heat conduction and stress-strain are enabled. A 50 s ramp to operating power is sufficient in practice.

7 Example 4: Gap Close and Reopen

File: `examples/fred_rod/gapclose_reopen/run.py`

This example exercises the gap-state logic: the fuel and cladding can come into physical contact (gap closes) and then separate again (gap reopens). Root-finding events in SUNDIALS IDA detect when the gap crosses the roughness threshold and automatically switches between the open-gap and closed-gap residual formulations.

Scenario (three phases):

1. **Phase 1 (0–100 s):** no power; open gap at the as-fabricated width of 15 µm.
2. **Phase 2 (100–600 s):** ramp to 5×10^8 W/m^3, then hold; fuel expands, gap closes.
3. **Phase 3 (600–1500 s):** ramp back to zero power; fuel contracts, gap reopens.

The as-fabricated gap is deliberately small (15 µm instead of the 100 µm in earlier examples) so that gap closure happens at a moderate power level.

Geometry with a tight as-fabricated gap.

```
g.rfo0 = 4.2e-3        # m
g.rci0 = 4.215e-3      # m  --- 15 µm as-fabricated gap
g.ruff = 5.0e-6        # fuel roughness
g.rufc = 5.0e-6        # cladding roughness
# contact threshold = ruff + rufc = 10 µm
```

The gap is considered *closed* when the mechanical gap width falls to the combined surface roughness ($r_{\text{uff}} + r_{\text{ufc}}$). In the closed state the pellet-cladding contact pressure is a free variable and the gap constraint is replaced by a no-penetration condition.

Multi-phase power history.

```
solver.set_power_density_history(
    [0.0, 100.0, 200.0, 600.0, 700.0, 1500.0],
    [0.0,   0.0, 5.0e8, 5.0e8,   0.0,    0.0])
```

Monitoring gap state.

```
gap_w = solver.gap_width()           # (n_steps,) m---axial average
pfc   = solver.contact_pressure()    # (n_steps,) MPa
rfo   = solver.fuel_outer_radius()   # (n_steps,) m
rci   = solver.clad_inner_radius()   # (n_steps,) m

# Gap-state classification
thresh = g.ruff + g.rufc     # contact threshold in m
is_closed = gap_w <= thresh  # boolean array
```

Also available in the raw IDA state vector:

```
y_out = solver.y_out()      # (n_steps, neq)
gpres = y_out[:, 0]         # gas pressure (MPa)---first DAE variable
```

What to expect.

- **Phase 1:** gap $\approx$ 13.8 µm (as-fabricated gap minus initial thermal expansion at 700 K).
- **Gap closure at $\approx$ 150 s:** fuel expansion exceeds the 15 µm as-fabricated gap; contact pressure jumps from 0 to $\approx$ 19 MPa.
- **Gap reopening at $\approx$ 700 s:** power removed; fuel contracts; gas pressure exceeds contact pressure; gap reopens.
- Results compare closely with the legacy FRED Fortran code ($|\Delta\text{gap}| < 0.2$ µm, $|\Delta\text{pfc}| <$ 0.1 MPa over the whole 1500 s run).

Key learning point

The gas pressure evolves via the ideal-gas DAE (included automatically in the IDA state vector). It rises with temperature during Phase 2 and drops when the fuel cools. The gap reopens when the gas pressure exceeds the contact pressure, not simply when the fuel contracts below the cladding radius.

8 Example 5: Hot Start vs Cold Start

File: `examples/fred_rod/hot_start_demo/run.py`

Every earlier example began at a low reference temperature and let the real transient carry the rod up to its operating (steady-state) condition — either directly (Example 1) or via a manual power ramp used purely to keep `IDACalcIC` well-behaved (Example 3). `set_hot_start(True)` replaces that workaround: the solver runs a pseudo-time pre-march at the $t = 0$ boundary conditions *before* the real time loop starts, converges to steady state, then re-anchors $t = 0$ at that converged state. The first logged output point is then already at operating conditions.

Cold vs hot start.

```
def make_solver(hot_start: bool):
    # ... identical geometry/materials/BCs each time (solver state is
        not
    # resettable after run(), so build a fresh solver per call) ...
    solver = fred.FredRodSolver(g, fuel, clad, gap)
    solver.set_enable_heat_conduction(True)
    solver.set_enable_stress_strain(True)
    solver.set_hot_start(hot_start)          # False = cold start (
        default)
```

```
    solver.set_power_density_history([0.0, 1e6], [1.0e8, 1.0e8])  #
        full power at t=0
    return solver

s_cold = make_solver(hot_start=False)
s_cold.run(tend=200.0, dtout=5.0)

s_hot = make_solver(hot_start=True)
s_hot.run(tend=200.0, dtout=5.0)
```

Both runs use the identical instantaneous full-power step at $t = 0$ (no manual ramp needed — the hot-start pseudo-time march, and IDA's own initial-condition solve for a literal cold jump, both handle the resulting residual imbalance without difficulty for this rod).

What to expect.

- **Cold run:** starts at $T_0 = 293.15$ K, rises to the ≈ 832.4 K steady-state centreline temperature by $t \approx 15$ s.
- **Hot run:** already at ≈ 832.4 K at $t = 0$; the maximum deviation from that value over the full 200 s run is $\lesssim 2 \times 10^{-7}$ K — i.e. "no change", confirming the pre-march genuinely found the same steady state the cold run converges to.

When *not* to use hot start

`heat_conduction_only` (Example 1) exists specifically to show the *transient* heat-up over several thermal time constants — enabling hot start there would erase the behaviour the example demonstrates. More generally, do not use hot start for any example whose purpose is to observe a startup transient, a gap-closure event under power cycling (Example 4), or a restart/checkpoint comparison against a known reference trajectory (Examples 6–7): hot start changes the $t = 0$ state itself, so it is only appropriate when steady state *before* the interesting behaviour begins is what you want as your starting point.

Key learning point

Hot start is a numerical convenience, not new physics: FRED-ROD has no irradiation state to freeze during the pre-march (unlike FRED-OX/FRED-M-Na, Section 18, where irradiation physics is explicitly disabled during the march). Use it whenever an example's own transient is not the point, to avoid burning simulated time (and, for stiffer coupled thermo-mechanical runs, solver robustness) on a ramp-up you do not care about.

9 Example 6: HDF5 Output

File: `examples/fred_rod/hdf5_output/run.py`

This example demonstrates how to write results to an HDF5 file using the `fred_input` / `fred_output` helper modules (located in `fred_platform/python/`) and how to read them back for post-processing.

Using `fred_input` instead of `fred_rod`. For scripts that write HDF5 output, import the validated Python wrapper:

```
import fred_input as fred    # validated wrapper; drop-in replacement
    for fred_rod
import fred_output as fo     # HDF5 read/write utilities
```

`fred_input` re-exports all the same classes (`FuelRodGeometry`, `MOX`, `T91`, `He`, `FredRodSolver`, ...) but validates all inputs in Python before forwarding them to the C++ layer, giving clearer error messages.

Running with HDF5 output.

```
solver.run(tend=1000.0, dtout=50.0, output_file='results.h5')
```

The `output_file` argument triggers `fred_output.write_results_h5()` immediately after `run()` completes. All results (temperatures, mechanical state, IDA restart vectors) are written to the specified HDF5 file.

Two output modes.

```
# Mode 1: dtout mode---results at every multiple of dtout
solver.run(tend=1000.0, dtout=50.0, output_file='results.h5')

# Mode 2: all_steps mode---results at every internal IDA step
solver2.run(tend=100.0, dtout=100.0, output_file='debug.h5', all_steps
    =True)
```

`all_steps=True` records every adaptive time step taken by IDA, which can be hundreds or thousands of points per second of simulated time. This mode is useful for diagnosing solver behaviour but produces large files.

HDF5 file layout. The file written by `write_results_h5()` has the following structure:

```
results.h5
  /metadata           -- attributes: app, nz, nf, nc, n_steps,
     all_steps
  /time               -- [n_steps]                    seconds
  /thermal/
      T               -- [n_steps, nz, nf+nc]         K
      peak_T_fuel     -- [n_steps]                    K
      gap_width       -- [n_steps]                    m  (axial avg)
  /mechanical/
      pfc             -- [n_steps]                    MPa
      rfo             -- [n_steps]                    m
      rci             -- [n_steps]                    m
      rco             -- [n_steps]                    m
      sigh_outer      -- [n_steps]                    MPa
  /restart/
      y               -- [n_steps, neq]               IDA state
      yp              -- [n_steps, neq]               IDA derivatives
```

Reading results back.

```
r = fo.read_results_h5('results.h5')

# Metadata
nz, nf, nc = r['nz'], r['nf'], r['nc']

# Arrays
times  = r['time']            # [n_steps] s
T      = r['T']               # [n_steps, nz, nf+nc] K
peak_T = r['peak_T_fuel']     # [n_steps] K
gap_w  = r['gap_width']       # [n_steps] m
pfc    = r['pfc']             # [n_steps] MPa
sigh   = r['sigh_outer']      # [n_steps] MPa
```

```
# Access radial temperature profile at final time, first axial layer:
T_profile = T[-1, 0, :]       # shape (nf+nc,)
T_fuel    = T[-1, 0, :nf]     # fuel nodes
T_clad    = T[-1, 0, nf:]     # cladding nodes

# IDA restart vectors (used by load_checkpoint / load_snapshot):
y_last  = r['restart_y'][-1, :]   # y-vector at final step
yp_last = r['restart_yp'][-1, :]
```

Key learning point

Storing the IDA `y`/`yp` vectors in the HDF5 file enables post-run analysis: you can reconstruct the solver at any saved output time and inspect or restart from that state. The `restart_snapshot` example (Section 10) shows a more structured API for this.

10 Example 7: Restart and Snapshot

File: `examples/fred_rod/restart_snapshot/`

This example contains three scripts that illustrate two complementary persistence mechanisms:

Checkpoint A single overwriting file for fault recovery. Simulation time is preserved on reload.

Snapshot A permanent restart-state file. Physical state (temperatures, strains, gap) is preserved but simulation time is reset to 0. Used to start a new transient from a preconditioned fuel state.

Script 0: Full reference run. `0_full_run.py` runs the complete simulation from $t = 0$ to $t = 5000$ s with both checkpointing and snapshot saving enabled.

```
WORK_DIR = 'ckpt'
tend   = 5000.0  # s
t_half = 3000.0  # s---mid-point snapshot

s = make_solver()

# Enable fault-recovery checkpoint (overwritten every dtout)
s.set_checkpoint_prefix(os.path.join(WORK_DIR, 'rod'))

# Save a snapshot at t_half; the final snapshot is always saved
    automatically
s.set_snapshot_prefix(os.path.join(WORK_DIR, 'rod'),
                      snapshot_times=[t_half])

s.run(tend, dtout=1000.0)
```

After running, the `ckpt/` directory contains:

```
rod_checkpoint.chk      # fault-recovery checkpoint (overwritten at
    each dtout)
rod_frame1.snapshot     # snapshot saved at t_half = 3000 s
rod_frame2.snapshot     # snapshot saved at tend   = 5000 s  (automatic)
ref_times.npy           # reference output times (for comparison in
    scripts 1 & 2)
ref_T.npy               # reference peak fuel temperatures
ref_gap.npy             # reference gap widths
```

Script 1: Checkpoint restart. `1_checkpoint_restart.py` demonstrates fault recovery. It simulates a crash at `t_half` by running only to that point, then reloads the checkpoint and continues to `tend`.

```
# Step A: partial run 0 → t_half (crashes here)
s_partial = make_solver()
s_partial.set_checkpoint_prefix(ckpt_prefix)
s_partial.run(t_half, dtout)

# Step B: reload checkpoint and continue to tend
s_restart = make_solver()
s_restart.set_checkpoint_prefix(ckpt_prefix)
s_restart.load_checkpoint(ckpt_file)
print(f"Restart time: {s_restart.restart_time():.0f} s")

s_restart.run(tend, dtout)
```

`restart_time()` returns the time from which the simulation will continue ($= t_{\text{half}}$ in this case). Results from the restarted run match the reference run to within 10^{-6} relative error on all quantities.

Script 2: Continuation via snapshot. `2_continuation.py` demonstrates the physical-state snapshot for a two-phase simulation: an "irradiation" phase followed by a "transient" phase with fresh time coordinates.

```
# Phase 1: irradiation, 0 → t_half
s1 = make_solver()
s1.set_snapshot_prefix(snap_prefix)    # saves rod_frame1.snapshot at
    end
s1.run(t_half, dtout)

# Phase 2: fresh start from the irradiated state
s2 = make_solver(t_end=t_phase2)
s2.load_snapshot(snap_file)
print(f"restart_time = {s2.restart_time():.1f} s")  # prints 0.0---
    time reset!

s2.run(t_phase2, dtout)

# Stitch results: Phase 2 times are offset by t_half for plotting
t_combined = np.concatenate([t1, t2[1:] + t_half])
```

The key difference between checkpoint and snapshot is the time-reset:

- `load_checkpoint()` → `restart_time()` returns the saved physical time; the run continues from there.
- `load_snapshot()` → `restart_time()` returns 0.0; the run restarts from $t = 0$ with the pre-irradiated fuel state.

Key learning point

Snapshots are the FRED-ROD mechanism for multi-stage simulations: you can build up a library of pre-conditioned fuel states (different burnup levels, gap sizes, strain states) and quickly start transient calculations from each one without re-running the irradiation phase.

11 Example 8: Variable Output Intervals (`dtout` as a Vector)

Files: `examples/fred_rod/dtout_example/{run_scalar,run_vector}.py`

`run(tend, dtout)` accepts `dtout` either as a scalar (constant output interval — every example so far) or as a *vector* of successive intervals (Section 2). This example runs the same coupled thermo-mechanical rod as Example 3 both ways and validates, in postprocessing, that the solver produced output at exactly the requested times.

Scalar (`run_scalar.py`).

```
solver.run(tend=200.0, dtout=10.0)      # t = 0, 10, 20, ..., 200 s

times    = np.asarray(solver.time_points())
expected = np.arange(0.0, 200.0 + 5.0, 10.0)
# validation: max |times - expected| and max |diff(times) - diff(
    expected)|
```

Vector (`run_vector.py`).

```
# very fine while the power ramp hits, coarse at steady state
# (sums to 205 s >= tend; np.sum(dtout) >= tend is enforced)
schedule = [0.1]*10 + [0.5]*8 + [2.0]*10 + [25.0]*7
solver.run(tend=200.0, dtout=schedule)

times    = np.asarray(solver.time_points())
expected = np.concatenate([[0.0], np.cumsum(schedule)])
expected = expected[expected <= 200.0 + 1e-9]
```

Both scripts print a PASS/FAIL report (both intervals and absolute times match the request to machine precision), `run_vector.py` additionally checks that an under-covering schedule (`np.sum(dtout) < tend`) raises `ValueError`, and a comparison plot (`dtout_validation.png`) shows the two runs' output spacing and their overlaying peak-fuel-temperature trajectories.

Why force small early intervals? For FRED-ROD/FRED-OX every output time is an IDA stop time, so a fine early schedule also *bounds the solver's internal step* through the stiffest part of a run (power step, contact onset) — a useful lever against numerical instabilities that is cheaper than shrinking `dtout` globally. Once past the transient, the intervals grow and the run costs no more than a constant-`dtout` one.

Same feature in FRED-M-Na

`examples/fred_m_na/dtout_example/` contains the equivalent scalar/vector pair for FRED-M-Na. There the effect is even more direct: the fixed backward-Euler integrator takes its internal step from the output interval whenever no explicit `set_step_size()` is given, so the vector schedule refines the *physics* steps themselves (visible in the run banner: `fixed dt=8640 s` during the 0.1-day entries vs. `86400 s` in the scalar run).

12 Example 9: Python-Defined Custom Materials

File: `examples/fred_rod/python_materials/run.py`

FRED-ROD exposes its abstract material interfaces to Python via `pybind11` trampoline classes. Any class that inherits from `fred.FuelPelletMaterial` or `fred.CladdingMaterial` and overrides the required methods can be passed directly to `FredRodSolver`. No C++ recompilation is needed.

This is the recommended approach for rapid prototyping of new material correlations before promoting them to a permanent C++ implementation.

12.1 Subclassing `FuelPelletMaterial`

```
import fred_rod as fred

T_REF = 293.15   # K---reference temperature for thermal expansion

class GenericOxideFuel(fred.FuelPelletMaterial):
    """
    Simplified oxide fuel with analytic correlations.
    All units follow the FRED convention (K, MPa, kg/m3, W/mK, J/kgK).
    """

    def thermalConductivity(self, T):
        """k(T) = 1/(A + B*T) + C*T^3  [W/(m·K)]"""
        A, B, C = 0.045, 2.8e-4, 5.6e-11
        return 1.0 / (A + B * T) + C * T**3

    def heatCapacity(self, T):
        """cp(T) = 260 + 0.10*T  [J/(kg·K)]"""
        return 260.0 + 0.10 * T

    def thermalExpansionStrain(self, T):
        """Linear CTE = 1e-5 /K"""
        return 1.0e-5 * (T - T_REF)

    def youngsModulus(self, T, density):
        """E0 = 200 GPa, linear softening  [MPa]"""
        return max(1.0e4, 2.0e5 * (1.0 - 1.5e-4 * (T - T_REF)))

    def poissonRatio(self):
        return 0.30

    def referenceDensity(self):
        return 10500.0     # kg/m^3  (as-fabricated, ~95.8% TD)

    def theoreticalDensity(self):
        return 10960.0     # kg/m^3

    def meltingTemperature(self):
        return 3100.0      # K
```

The `thermalConductivity(T)` form $k = 1/(A+BT)+CT^3$ is a standard two-term expression for oxide ceramics: the first term captures phonon scattering (decreasing with temperature) and the second term captures the small-polaron contribution (increasing with temperature).

12.2 Subclassing `CladdingMaterial`

```
class GenericSteelClad(fred.CladdingMaterial):

    def thermalConductivity(self, T):
        """k(T) = 23.5 + 0.005*(T - 273.15)  [W/(m·K)]"""
        return 23.5 + 0.005 * (T - 273.15)

    def heatCapacity(self, T):
```

```
        """cp(T) = 480 + 0.12*T  [J/(kg·K)]"""
        return 480.0 + 0.12 * T

    def thermalExpansionStrain(self, T):
        """CTE = 11.5e-6 /K"""
        return 11.5e-6 * (T - T_REF)

    def youngsModulus(self, T):
        """Linear softening from 200 GPa  [MPa]"""
        return max(1.0e4, 2.0e5 - 60.0 * (T - T_REF))

    def poissonRatio(self):
        return 0.30

    def meyerHardness(self, T):
        """H_M(T) = 5e9 * T^(-0.2)  [Pa]"""
        return 5.0e9 * T**(-0.2)

    def referenceDensity(self):
        return 7800.0    # kg/m^3
```

Note that `CladdingMaterial.youngsModulus(T)` takes only temperature (no density), whereas `FuelPelletMaterial.youngsModulus(T, density)` takes both (porosity affects the fuel modulus). `meyerHardness(T)` is used in the Ross–Stoute solid contact conductance model when the gap is closed.

12.3 Required abstract methods

Table 1 lists every method that *must* be overridden. FRED will raise a `RuntimeError` if any method is not implemented.

Table 1: Abstract methods that must be implemented in Python subclasses.

Class	Method	Returns
`FuelPelletMaterial`	`thermalConductivity(T)`	W/(m·K)
	`heatCapacity(T)`	J/(kg·K)
	`thermalExpansionStrain(T)`	— (strain, zero at T_{ref})
	`youngsModulus(T, density)`	MPa
	`poissonRatio()`	—
	`referenceDensity()`	kg/m^3
	`theoreticalDensity()`	kg/m^3
	`meltingTemperature()`	K
`CladdingMaterial`	`thermalConductivity(T)`	W/(m·K)
	`heatCapacity(T)`	J/(kg·K)
	`thermalExpansionStrain(T)`	—
	`youngsModulus(T)`	MPa
	`poissonRatio()`	—
	`meyerHardness(T)`	Pa
	`referenceDensity()`	kg/m^3

12.4 Using the custom materials in a simulation

```
fuel = GenericOxideFuel()
```

```
clad = GenericSteelClad()
gap  = fred.DummyGapMaterial()

solver = fred.FredRodSolver(g, fuel, clad, gap)  # standard API
solver.set_enable_heat_conduction(True)
solver.set_enable_stress_strain(True)

# ... set BCs and run as usual ...
solver.run(tend=1000.0, dtout=100.0)
```

There is no difference in how `FredRodSolver` is used: the Python objects are polymorphically identical to the built-in C++ materials from the solver's perspective.

12.5 Verifying your material with the built-in API

You can call the material methods directly from Python to verify they are correct before running a full simulation:

```
fuel = GenericOxideFuel()

for T in [400, 700, 1000, 1400]:
    k  = fuel.thermal_conductivity(T)
    cp = fuel.heat_capacity(T)
    print(f"T={T:4d} K: k={k:.3f} W/mK, cp={cp:.1f} J/kgK")
```

Python call overhead

Each Python material method is called by the C++ solver once per radial node per Newton iteration per time step. For a run with thousands of time steps and fine meshes this can introduce measurable overhead compared to a compiled C++ material. Once a correlation is validated, implement it in C++ and add pybind11 bindings following the pattern in `src/apps/fred_rod/fuelpelletmaterial/UO2.hpp/.cpp` and `bindings.cpp`.

13 Example 10: Advanced Prototyping — Python to C++

Files: `examples/fred_rod/advanced_prototyping/`

Section 12 showed how to prototype fuel and cladding materials in Python. The same mechanism applies to gap materials via `fred.GapMaterial`. This example uses a physically motivated model — a burnup-dependent He/Kr/Xe gap conductivity — to demonstrate a two-step development loop:

1. **Step 1** (`run_python.py`): implement the model as a Python subclass. No rebuild required; changes take effect immediately.

2. **Step 2** (`run_cpp.py`): use the compiled C++ equivalent `fred.HeKrXe`. Results are numerically identical; C++ removes Python call overhead in the inner solver loop.

13.1 Physics: fission gas release and gap conductivity

Fresh fuel pins are filled with helium (excellent thermal conductor, $k_{\mathrm{He}} \approx 0.27$ W/mK at 700 K). During irradiation, xenon and krypton — fission products with conductivities 10–20× lower than helium — are released into the gap gas. Even a moderate fission gas release (FGR) fraction significantly raises the fuel temperature.

The model implemented here uses three steps:

FGR fraction (Waltar-Reynolds, restructured zone):

$$B_{\mathrm{MWd}} = B_{\mathrm{at\%}} \times 9.4 \qquad (1\,\mathrm{at\%} \approx 9.4\,\mathrm{MWd/kgHM})$$
$$\mathrm{FGR} = \max\!\left(0,\; 1 - \frac{4.7}{B_{\mathrm{MWd}}}\left(1 - e^{-B_{\mathrm{MWd}}/5.9}\right)\right) \tag{3}$$

Molar balance:

$$n_{\mathrm{He},0} = \frac{P_{\mathrm{fill}} V_{\mathrm{free}}}{R T_{\mathrm{ref}}} \approx 6.15 \times 10^{-5}\,\mathrm{mol} \quad (P = 0.1\,\mathrm{MPa},\; V = 1.5\,\mathrm{cm}^3,\; T = 293\,\mathrm{K})$$
$$n_{\mathrm{FG}} = \mathrm{FGR} \times 0.25 \times \frac{B_{\mathrm{at\%}}}{100} \times \frac{m_{\mathrm{fuel}}}{M_{\mathrm{HM}}}$$
$$y_{\mathrm{He}} = (n_{\mathrm{He},0} + 0.0385\, n_{\mathrm{FG}}) \;/\; n_{\mathrm{total}}$$
$$y_{\mathrm{Kr}} = 0.0769\, n_{\mathrm{FG}} \;/\; n_{\mathrm{total}}, \qquad y_{\mathrm{Xe}} = 0.8846\, n_{\mathrm{FG}} \;/\; n_{\mathrm{total}} \tag{4}$$

Geometric-mean mixture conductivity (FRED.f90 `gaphtc`):

$$k_{\mathrm{mix}}(T) = k_{\mathrm{He}}^{y_{\mathrm{He}}}\, k_{\mathrm{Kr}}^{y_{\mathrm{Kr}}}\, k_{\mathrm{Xe}}^{y_{\mathrm{Xe}}}, \qquad k_i(T) = A_i\, T^{B_i} \tag{5}$$

Typical values at 700 K and several burnup levels:

B [at%]	FGR [%]	y_{He}	y_{Kr}	y_{Xe}	k_{mix} [W/mK]
0.0	0	1.000	0.000	0.000	0.274
1.0	60	0.513	0.039	0.448	0.062
5.0	90	0.149	0.068	0.783	0.020

13.2 Step 1: Python prototype (`run_python.py`)

The `GapMaterial` interface requires a single method:

```
import fred_rod as fred
import math

class HeKrXePy(fred.GapMaterial):

    _A_He, _B_He = 2.639e-3, 0.7085    # k_He(T) = A * T^B  [W/(m·K)]
    _A_Kr, _B_Kr = 8.247e-5, 0.8363
    _A_Xe, _B_Xe = 4.351e-5, 0.8618

    def __init__(self, bup_atpct=0.0):
        super().__init__()
        bup_MWd = bup_atpct * 9.4
        fgr = 0.0
        if bup_MWd > 0.0:
            a   = 4.7 / bup_MWd * (1.0 - math.exp(-bup_MWd / 5.9))
            fgr = max(0.0, 1.0 - a)

        n_He0  = 0.1e6 * 1.5e-6 / (8.314 * 293.15)   # ~6.15e-5 mol
        fggen  = 0.25 * (bup_atpct / 100.0) * 0.010 / 0.238
        fgrel  = fgr * fggen
        n_Xe   = 0.8846 * fgrel
        n_Kr   = 0.0769 * fgrel
        n_HeFG = 0.0385 * fgrel
        n_tot  = n_He0 + n_HeFG + n_Kr + n_Xe
```

```
        self.y_He = (n_He0 + n_HeFG) / n_tot
        self.y_Kr = n_Kr / n_tot
        self.y_Xe = n_Xe / n_tot

    def gapConductivity(self, T):
        k_He = self._A_He * T ** self._B_He
        k_Kr = self._A_Kr * T ** self._B_Kr
        k_Xe = self._A_Xe * T ** self._B_Xe
        return k_He ** self.y_He * k_Kr ** self.y_Kr * k_Xe ** self.
            y_Xe
```

The platform takes `gapConductivity(T)` and converts it to a gap conductance internally using the Lanning-Hann gas-jump model with the actual (time-varying) gap width and gas pressure. The material class does not need to know the gap width.

`GapMaterial` interface

Only `gapConductivity(T)` must be overridden. The solver calls it at each Newton iteration to obtain k [W/(m·K)], then computes the full gap conductance h [W/(m^2·K)] using the gap geometry. Optionally override `isGasBond()` (`True` by default) and `clampConductance(h, gap_closed)` for liquid-bond materials (sodium, lead) where the conversion from k to h is different.

Three simulations are run with this gap material at increasing burnup levels, with UO2 fuel, AIM1 cladding, and a constant volumetric power of 3×10^8 W/m^3:

Burnup	T_{peak} [K]	Gap [μm]	$\sigma_{\theta,\text{outer}}$ [MPa]
0.0 at% (as-fabricated)	1272	81.8	22.3
1.0 at% (moderate)	1678	63.4	22.3
5.0 at% (high)	2008	46.1	22.4

The +735 K rise from 0 to 5 at% illustrates the strong coupling between fission gas release and peak fuel temperature.

13.3 Step 2: Compiled C++ equivalent (`run_cpp.py`)

Once the Python model is validated, use the compiled `fred.HeKrXe` class — which implements the same physics in C++:

```
gap = fred.HeKrXe(bup_atpct=5.0)    # compiled class, same physics
s   = fred.FredRodSolver(geom, fred.UO2(), fred.AIM1(), gap)
# ... identical setup and run() call ...
```

The `run_cpp.py` script also verifies that the conductivity values agree with the Python prototype to machine precision before running the simulations.

13.4 Adding your own compiled gap material

To follow the same Python→C++ path for your own model:

1. Write a header-only class in `src/apps/fred_rod/gapmaterial/MyGap.hpp` that inherits `fred::GapMaterial` and overrides `gapConductivity(double T) const`.
2. Add a pybind11 binding block in `src/apps/fred_rod/bindings.cpp`:

```cpp
#include "apps/fred_rod/gapmaterial/MyGap.hpp"
// inside bind_fred_rod():
py::class_<MyGap, GapMaterial>(m, "MyGap")
    .def(py::init<double>(), py::arg("param") = 0.0,
         "Description of MyGap.");
```

3. Rebuild: `make fred-rod`. No `Makefile` source-list change is needed for header-only classes.

See `examples/fred_rod/advanced_prototyping/README.md` for the complete walkthrough, including the `.cpp`-file variant and liquid-bond material instructions.

14 FRED-ROD API Reference

14.1 Geometry: `FuelRodGeometry`

Attribute	Type	Description
`nf`	int	Fuel radial nodes (≥ 2)
`nc`	int	Cladding radial nodes (≥ 2)
`nz`	int	Axial layers (≥ 1)
`rfi0`	float	Inner fuel radius [m] (0 = solid pellet)
`rfo0`	float	Outer fuel radius [m]
`rci0`	float	Inner cladding radius [m]
`rco0`	float	Outer cladding radius [m]
`dz0`	list	Axial layer heights [m] (length `nz`)
`vgp`	float	Gas plenum volume [m^3]
`ruff`	float	Fuel outer surface roughness [m]
`rufc`	float	Cladding inner surface roughness [m]

14.2 Built-in materials

Class	Type	Notes
`UO2(reference_density)`	Fuel	Philipponneau conductivity
`MOX(pu_content, [ref_density])`	Fuel	Philipponneau; $x_{\mathrm{Pu}} \in [0.01, 0.5]$
`DummyFuelPellet()`	Fuel	Constant $k = 10$, $\rho = 10000$, $c_p = 100$
`AIM1([reference_density])`	Cladding	Austenitic SS (Luzzi correlations)
`T91([reference_density])`	Cladding	Ferritic-martensitic (KIT)
`DummyCladding()`	Cladding	Constant $k = 10$, $\rho = 10000$, $c_p = 100$
`He()`	Gap	Pure helium
`HeKrXe(bup_atpct)`	Gap	He-Kr-Xe mixture; composition from burnup
`DummyGapMaterial()`	Gap	$h(T) = 500 + 2.5T$ [W/m^2K]

14.3 Solver methods: `FredRodSolver`

Method	Description
`set_enable_heat_conduction(bool)`	Toggle thermal ODE
`set_enable_stress_strain(bool)`	Toggle mechanics
`set_initial_temperature(T0_K)`	Uniform IC [K]
`set_initial_gas_pressure(p0_MPa)`	Fill-gas pressure IC [MPa]
`set_coolant_temperature(times, T_K)`	Outer BC history [K]
`set_coolant_pressure(p_MPa)`	Constant coolant pressure [MPa]
`set_coolant_pressure_history(t, p)`	Time-varying coolant pressure [MPa]
`set_power_density_history(times, qv)`	Volumetric heat [W/m^3]
`set_tolerances(rtol, atol)`	IDA tolerances
`set_hot_start(bool)`	Pseudo-time pre-march to steady state before $t = 0$ (Section 8)
`set_checkpoint_prefix(prefix)`	Enable fault-recovery checkpoint
`set_snapshot_prefix(prefix, [times])`	Enable physical-state snapshots
`load_checkpoint(filename)`	Load checkpoint (time preserved)
`load_snapshot(filename)`	Load snapshot (time reset to 0)
`set_output_file(filename)`	Stream results to HDF5 during `run()` (native path)
`run(tend, dtout, [all_steps])`	Execute simulation
`time_points()`	Output times [s]
`temperatures()`	Nodal temperatures, shape `(n,nz*(nf+nc))` [K]
`peak_fuel_temperature()`	Peak fuel temperature [K]
`gap_width()`	Axial-avg gap width [m]
`contact_pressure()`	Axial-avg contact pressure [MPa]
`clad_outer_hoop_stress()`	Axial-avg outer clad hoop stress [MPa]
`clad_outer_radius()`	Axial-avg outer cladding radius [m]
`fuel_outer_radius()`	Axial-avg fuel outer radius [m]
`clad_inner_radius()`	Axial-avg cladding inner radius [m]
`y_out()`	IDA state vector, shape `(n,neq)`
`yp_out()`	IDA derivative vector, shape `(n,neq)`
`restart_time()`	Restart time from last load [s]

15 FRED-ROD Troubleshooting

`IDACalcIC` **failure at $t = 0$ with heat+stress-strain enabled.** Either use a slow power ramp (at least 50 s) instead of a step at $t = 0$ (see Section 6), or — if a startup transient is not what you want to observe anyway — switch to `set_hot_start(True)` (Section 8), which sidesteps the issue entirely by pre-converging before $t = 0$.

Gap does not close when expected. Check that `rci0 - rfo0` is small enough relative to the operating temperature difference, and that `ruff + rufc` (the contact threshold) is less than the as-fabricated gap.

Custom Python material causes segfault. Ensure your subclass does not raise exceptions inside the overridden methods: C++ does not handle Python exceptions gracefully. Validate the correlation range in Python before passing the material to the solver.

HDF5 file not created. The `output_file` argument to `run()` requires `h5py` to be installed (`pip install h5py`).

`fred_input` **or** `fred_output` **not found.** These modules live in `fred_platform/python/`. Either add that directory to your `PYTHONPATH`, copy the files to your working directory, or add `sys.path.insert(0, '/path/to/fred_platform/python')` to your script.

Part II
FRED-OX

FRED-OX uses the same coupled heat-conduction / thermo-elastic stress-strain equations as FRED-ROD (Part I) and adds oxide-fuel-specific irradiation physics on top: empirical base-irradiation correlations for fuel/cladding properties, and fission-gas production/release, which feed back into gas-plenum pressure (an extra DAE state) and gap conductance. Where FRED-ROD leaves irradiation modelling entirely to the user, FRED-OX provides it built in for oxide (UO_2/MOX) fuel.

The solver class is `FredOxSolver` (module `fred_ox` / compiled extension `_fred_ox`), used in place of `FredRodSolver`, with FRED-OX-specific material classes: `FredOxMOX(pu_content, rof0, sto0)`, `FredOxT91(reference_density)`, `FredOxGapMaterial()`. Geometry is still built with the same `FuelRodGeometry`.

16 Recommended Example Order

The example scripts under `examples/fred_ox/` are best worked through in the following order:

1. `base_irradiation` — first FRED-OX run: thermal + base-irradiation + fission-gas-release physics, validated against the legacy FRED Fortran code.
2. `hot_start_demo` — cold vs hot start for FRED-OX, with irradiation physics explicitly frozen during the pre-march.
3. `hdf5_output` — the FRED-OX-specific HDF5 fields (gas pressure, fission gas, burnup) on top of temperature.
4. `restart` — checkpoint-based fault recovery, carrying irradiation state (burnup, gas inventory) through a restart.
5. `restart_snapshot` — checkpoint *and* named-snapshot workflows combined.
6. `ESFR_pin` — the most advanced example: a 600-day base-irradiation benchmark followed by two fast transients (ULOF, UTOP) loaded from a saved irradiated snapshot.

Why this order?

`base_irradiation` is the FRED-OX analogue of FRED-ROD's Examples 1–4: it is the first place you see the extra irradiation state (burnup, fission gas, gas pressure) and the per-layer (axially resolved) boundary conditions that most real FRED-OX runs use. `hot_start_demo` builds directly on it (same style of pin, shorter run) and is easiest to understand once you know what "irradiation effects" means for FRED-OX. `hdf5_output`, `restart`, and `restart_snapshot` mirror the equivalent FRED-ROD examples (Sections 9, 10) but are worth revisiting for OX specifically because irradiation state (burnup, gas inventory) must persist correctly across restarts, not just temperatures and stresses. `ESFR_pin` is last because it combines everything: a long base-irradiation run, snapshotting, and transient analysis starting from a realistically irradiated state.

17 Example OX-1: Base Irradiation

File: `examples/fred_ox/base_irradiation/irradiate_10d/run.py`

This is the FRED-OX equivalent of FRED-ROD's Example 3 (Section 6): heat conduction and stress-strain are both active, but now base-irradiation correlations and fission-gas release are active too, and boundary conditions are specified *per axial layer* rather than as single rod-wide histories.

Scenario: an ESFR-SMART-style pin, `nz=20` axial layers (`dz=0.05 m` each, 1.0 m active length), `nf=22` fuel nodes, `nc=5` cladding nodes. MOX fuel (`pu_content=0.1799`, `rof0=10542 kg/m^3`), T91 cladding (`reference_density=7790 kg/m^3`); `rfi0=1.56e-3 m`, `rfo0=4.68e-3 m`, `rci0=rfo0+1.55e-4 m`, `rco0=5.335e-3 m`, `vgp=7.19e-5 m^3`. Power ramps from 0 to a per-layer peak of 6.934×10^8 W/m^3 (layer 12) between $t = 3600$ s and $t = 86400$ s; clad outer temperature ramps from 668.0 K up to 824.31 K over the same window; `coolant_pressure=0.2 MPa`, `initial_gas_pressure=0.1 MPa`; `tend=86400 s`, `dtout=1800 s`; tolerances `(1e-5, 1e-7)`.

Per-layer boundary conditions.

```
solver.set_power_density_history_per_layer(times, qqv_per_layer)    # W
    /m3, one column per layer
solver.set_coolant_temperature_per_layer(times, Tco_per_layer)      # K
solver.set_plenum_temperature_history(times, T_plenum)              # K
solver.set_swelling_multiplier(0.8)    # scales solid fission-product
    swelling
```

`set_power_density_history_per_layer` and `set_coolant_temperature_per_layer` take a 2-D array (time points × `nz`) instead of the single-column history used in FRED-ROD; this is what lets the axial power/temperature profile vary along the pin. `set_plenum_temperature_history` is new: it feeds the gas-plenum temperature used in the gas-pressure DAE.

New result accessors.

```
gpres = solver.gas_pressure()           # (n_steps,) MPa   -- plenum gas
    pressure
fggen = solver.fg_generated()           # (n_steps,) mol   -- total
    fission gas generated
fgrel = solver.fg_released()            # (n_steps,) mol   -- total
    fission gas released
bup   = solver.burnup()                 # (n_steps,) MWd/kgU
gap_w = solver.gap_width()              # (n_steps,) m, axial average
Tpeak = solver.peak_fuel_temperature() # (n_steps,) K
```

Legacy comparison at $t = 86400$ s.

Quantity	FRED-OX	Legacy FRED (Fortran)
Gas pressure [MPa]	0.24326	—
Fission gas generated [cm^3 STP]	0.33828	—
Fission gas released [cm^3 STP]	0.013437	—
FGR fraction [%]	3.972	(reference: `outfrd000000000048`)

Gas amounts are stored in moles internally and converted to cm^3 at STP for comparison with the legacy output via $V_{\mathrm{STP}} = n \times 8.314 \times 293.15/10^5 \times 10^6$. The script produces `plots/gas_pressure.png`, `plots/peak_T_fuel.png`, `plots/fission_gas.png`, and `plots/gap_width.png`.

Key learning point

`set_swelling_multiplier` scales only the *solid* fission-product swelling correlation — a tuning knob for matching a specific fuel design's densification/swelling behaviour. It does not affect gas-bubble swelling or fission-gas release, which are computed from the burnup history directly.

18 Example OX-2: Hot Start vs Cold Start

File: `examples/fred_ox/hot_start_demo/run.py`

The same `set_hot_start(True)` mechanism introduced for FRED-ROD (Section 8), but here the pseudo-time pre-march explicitly **disables irradiation physics** (fission gas release, swelling, burnup-dependent conductivity) while it converges the thermal/mechanical state, then re-enables irradiation and re-anchors $t = 0$. This matters because FRED-OX's irradiation state is time-integrated (burnup, gas inventory) — letting it accumulate during an artificial pseudo-time march would give a physically meaningless burnup at "$t = 0$".

Scenario: a smaller single-layer MOX pin (`nz=1, nf=10, nc=3`) so the demo runs fast: `rfo0=4.68e-3 m`, `rci0=rfo0+1.55e-4 m` (155 μm gap), `rco0=5.335e-3 m`, `vgp=7.19e-5 m`^{3}; power 2×10^8 W/m^3; coolant and plenum temperature both 700 K; `coolant_pressure=0.2 MPa`, `initial_gas_pressure=0.1 MPa`; `tend=300 s`, `dtout=10 s`; tolerances `(1e-5, 1e-7)`. 300 s is chosen deliberately: long enough for FRED-OX's ∼150–200 s thermal time constant to reach steady state, short enough that burnup and FGR stay negligible ($\mathrm{fgrel}, \mathrm{bup} \sim 10^{-4}$–$10^{-22}$, printed explicitly by the script) — i.e. "hot steady state, before any serious irradiation effects", matching the docstring's own framing.

What to expect.

- **Cold run:** $T_0 = 293.15$ K → peak fuel temperature ≈ 1454.83 K by $t \approx 150$ s.
- **Hot run:** flat at ≈ 1454.834 K for the whole run (max deviation from its own $t = 0$ value $\approx 1.95 \times 10^{-4}$ K).

Interpreting "no serious irradiation effects"

The script asserts `fgrel`/`bup` stay tiny over the 300 s window specifically to prove that the hot-vs-cold temperature comparison is a thermal-only effect, not an artifact of irradiation state diverging between the two runs. For a longer run (hours to days), do not assume the same holds — burnup accumulates in real time regardless of hot/cold start, so the two runs' irradiation states will diverge once real time passes $t = 0$, even though they started from the same thermal state.

19 Example OX-3: HDF5 Output

File: `examples/fred_ox/hdf5_output/run.py`

Extends FRED-ROD's HDF5 example (Section 9) with the FRED-OX-specific fields. Uses the same `fred_input`/`fred_output` pattern:

```
import fred_input as fred
import fred_output as fo

solver.run(tend=86400.0, dtout=1800.0, output_file='results.h5')
solver2.run(tend=7200.0, dtout=3600.0, output_file='debug.h5',
   all_steps=True)

r = fo.read_results_h5('results.h5')
gpres  = r['gpres']          # (n_steps,) MPa
fggen  = r['fggen']          # (n_steps,) mol
fgrel  = r['fgrel']          # (n_steps,) mol
bup    = r['bup']            # (n_steps,) MWd/kgU
gap_w  = r['gap_width']      # (n_steps,) m
peak_T = r['peak_T_fuel']    # (n_steps,) K
```

```
T      = r['T']              # (n_steps, nz, nf+nc) K -- per-layer,
    unlike ROD's flat array
```

Shape convention

Unlike FRED-ROD's single-layer examples, `r['T']` here is genuinely 3-D (`[n_steps, nz, nf+nc]`) because `nz>1`. The script uses this to plot an axial temperature profile (fuel centreline and clad outer surface vs. layer) at the final time step — not meaningful for the single-layer FRED-ROD examples.

Unit conventions for the new fields match Example OX-1: gas amounts in mol (convert to cm^3 STP the same way), burnup in MWd/kgU, gas pressure in MPa.

20 Example OX-4: Restart (Checkpoint)

File: `examples/fred_ox/restart/run.py`

Checkpoint-only fault recovery, the FRED-OX analogue of the checkpoint half of FRED-ROD's Example 7 (Section 10): a single overwritten binary file for resuming after a crash, with simulation time *preserved* on reload (as opposed to a snapshot, where time resets to 0 — see Example OX-5).

Scenario: a 3-day (`tend=3*86400 s`) single-layer MOX/T91 pin (`nf=4, nc=3, nz=1`), `pu_content=0.20`, `rof0=10200 kg/m`^{3}, cladding `reference_density=7750 kg/m`^{3}, power 1.5×10^8 W/m^3, coolant 620 K, `coolant_pressure=0.1 MPa`.

```
solver.set_checkpoint_prefix(ckpt_prefix)
solver.run(tend, dtout)                          # (a) full reference run

s_partial = make_solver()
s_partial.set_checkpoint_prefix(ckpt_prefix)
s_partial.run(86400.0, dtout)                    # (b) crash simulated at
    day 1

s_restart = make_solver()
s_restart.set_checkpoint_prefix(ckpt_prefix)
s_restart.load_checkpoint(ckpt_file)
print(s_restart.restart_time())                  # non-zero, e.g. ~86400
   s -- time preserved
s_restart.run(tend, dtout)                       # (c) continue to tend
```

The script then verifies that `peak_fuel_temperature()`, `gas_pressure()`, `burnup()`, and `fg_released()` from the restarted run match the full reference run at overlapping output times (`rtol=1e-6`, gas pressure `1e-5`), printing a PASS/FAIL comparison table — confirming that irradiation state, not just thermal state, survives a checkpoint round-trip exactly.

21 Example OX-5: Restart and Snapshot

Files: `examples/fred_ox/restart_snapshot/{0_full_run,1_checkpoint_restart,2_continuation}.py`

The same three-script pattern as FRED-ROD's Example 7 (Section 10), using the identical 3-day scenario as Example OX-4, but exercising *both* persistence mechanisms together and confirming FRED-OX's irradiation state (burnup, fission-gas inventory) survives each one correctly:

`0_full_run.py` Reference run. Enables both `set_checkpoint_prefix(prefix)` and `set_snapshot_prefix(pre`
`snapshot_times=[t_half])`. Produces `ox_checkpoint.chk` (overwritten fault-recovery

file), `ox_frame1.snapshot` (at $t_{\text{half}} = 86400$ s), `ox_frame2.snapshot` (final state, saved automatically), and reference `.npy` arrays (`ref_times/T/gp/bup/fgr.npy`).

`1_checkpoint_restart.py` Same pattern as Example OX-4: partial run to day 1 with a checkpoint, reload, `restart_time()` confirmed non-zero, continue to `tend`; verified against the script-0 reference.

`2_continuation.py` Snapshot-based two-phase run: Phase 1 ($0 \to t_{\text{half}}$) auto-saves a snapshot; Phase 2 calls `s2.load_snapshot(snap_file)`, confirms `s2.restart_time() == 0.0` (time reset) while burnup and fission-gas state are preserved from Phase 1, then runs $0 \to t_{\text{end}} - t_{\text{half}}$. Trajectories are stitched by offsetting Phase 2 times by t_{half}, and the combined series is verified against the script-0 reference.

Key learning point

The fields explicitly carried through and checked across every restart path here — `peak_fuel_temperature`, `gas_pressure`, `burnup`, `fg_released` — are exactly the extra DAE/ODE state FRED-OX adds over FRED-ROD. This example is the most direct proof that the gas-plenum-pressure DAE, burnup accumulator, and FGR inventory all persist correctly through both time-preserving (checkpoint) and time-resetting (snapshot) restarts.

22 Example OX-6: ESFR Pin — Base Irradiation and Transients

Files: `examples/fred_ox/ESFR_pin/{base_irradiation,ULOF_transient,UTOP_transient}/run.py`

The most advanced FRED-OX example: a full-length base-irradiation benchmark followed by two fast, unprotected-transient analyses that pick up from the irradiated state via a saved snapshot. ESFR = European Sodium Fast Reactor (ESFR-SMART); all three scripts share the same pin geometry and materials as Example OX-1 (`nz=20, nf=22, nc=5`, MOX/T91/He).

22.1 `base_irradiation`: 600-day benchmark

Same setup as Example OX-1 but run for 600 days ($t_{\text{end}} = 5.184 \times 10^7$ s, `dtout=86400 s`), with boundary conditions given at 5 knot times $(0, 3600, 86400, 2.592 \times 10^7, 5.184 \times 10^7$ s) per axial layer. At $t = 600$ d the legacy reference gives: gas pressure 0.59687 MPa, fission gas generated 428.871 cm^3 STP, released 101.969 cm^3 STP (23.776% FGR), peak fuel temperature (layer 13) 1826.6 °C. The new call here is:

```
solver.set_snapshot_prefix(prefix)    # auto-saves esfr_pin_base_frame1
    .snapshot at run end
```

This snapshot — the fully irradiated, 600-day pin state — is what the two transient scripts below load.

22.2 `ULOF_transient`: Unprotected Loss of Flow (43 s)

```
solver = make_solver()
solver.load_snapshot('../base_irradiation/snapshots/
    esfr_pin_base_frame1.snapshot')
print(solver.restart_time())           # 0.0 -- confirms time reset;
    irradiation state kept
solver.set_swelling_multiplier(0.0)    # freeze further solid swelling
    during the fast transient
solver.run(tend=43.0, dtout=0.1)       # 430 output steps
```

Models a coolant pump-halving event: 38 boundary-condition time points span $t = 0$ to 43.5 s (pump halving time 10 s; power drops ∼28% over the transient as flow-driven reactivity feedback kicks in). Tolerances `(1e-5, 1e-8)`. `set_swelling_multiplier(0.0)` is set because 43 s is far too short for any further irradiation-timescale solid swelling to occur — only the already-accumulated (loaded-from-snapshot) swelling and the fast thermal/mechanical response matter here.

22.3 `UTOP_transient`: Unprotected Transient Over-Power (1000 s)

Same snapshot-loading pattern as ULOF, but models a +280 pcm reactivity insertion: power peaks ∼37% above nominal, then decays, over 52 boundary-condition time points spanning $t = 0$ to 1000.4 s. Also uses `set_swelling_multiplier(0.0)`; `dtout=1.0 s` (1000 output steps); tolerances `(1e-4, 1e-8)`.

Both transient scripts print final gas pressure, peak fuel temperature, average gap width, and fission gas released, and produce a 2×2 subplot grid (`plots/ulof_results.png` / `plots/utop_results.png`) of these quantities vs. time.

Key learning point

`ESFR_pin` demonstrates the intended FRED-OX workflow for transient safety analysis: run (or load from a validated database) a long base-irradiation history to build up realistic burnup/gas/swelling state, snapshot it, then branch into as many fast transient scenarios as needed — each starting from the same irradiated state at $t = 0$ without re-running the multi-hundred-day irradiation each time.

23 FRED-OX API Reference

23.1 Materials

Class	Type	Notes
`FredOxMOX(pu_content, rof0, sto0)`	Fuel	MOX with base-irradiation correlations
`FredOxT91(reference_density)`	Cladding	Ferritic-martensitic, irradiation-aware
`FredOxGapMaterial()`	Gap	He/fission-gas mixture, burnup-coupled

23.2 Solver methods: `FredOxSolver` — additions over `FredRodSolver`

Method	Description
`set_power_density_history_per_layer(t, qqv)`	Per-layer volumetric heat [W/m^3]
`set_coolant_temperature_per_layer(t, T)`	Per-layer coolant BC [K]
`set_plenum_temperature_history(t, T)`	Gas-plenum temperature [K]
`set_swelling_multiplier(f)`	Scale solid fission-product swelling
`set_hot_start(bool)`	Pre-march with irradiation frozen (Section 18)
`gas_pressure()`	Gas-plenum pressure [MPa] (extra DAE state)
`fg_generated()`	Total fission gas generated [mol]
`fg_released()`	Total fission gas released [mol]
`burnup()`	Rod burnup [MWd/kgU]

All other geometry attributes, materials-interface concepts, and solver methods (temperatures, gap width, checkpoint/snapshot, HDF5 output) are identical to FRED-ROD — see

Section 14.

24 FRED-OX Troubleshooting

Fission gas release seems to jump discontinuously across a restart. Confirm you used `load_checkpoint` (time-preserving) and not `load_snapshot` (time-resetting) if you intended to continue the *same* irradiation history — see Section 20. Loading a snapshot deliberately resets simulation time to 0 while keeping burnup/gas state, which is correct for starting a new transient from an irradiated pin, but wrong if you wanted a literal continuation.

`set_hot_start(True)` gives an unexpectedly large burnup at $t = 0$. This should not happen — irradiation physics is frozen during the pre-march (Section 18). If you see nonzero burnup growth at $t = 0$, check you are calling `set_hot_start` before `run()`, not after building a solver you have already partially run.

Per-layer boundary condition array shape errors. `set_power_density_history_per_layer` and `set_coolant_temperature_per_layer` expect a 2-D array shaped `(n_time_points, nz)` — one column per axial layer, in the same top-to-bottom order as `g.dz0`. A single-layer (`nz=1`) example still needs shape `(n, 1)`, not a flat 1-D array.

Legacy comparison mismatch after changing `set_swelling_multiplier`. The legacy reference values quoted in Examples OX-1 and OX-6 assume the default multiplier used in those scripts (0.8 for `base_irradiation`, unset/1.0 elsewhere). Changing it is a deliberate tuning choice and will shift gap width, gas pressure, and hence FGR relative to the quoted reference numbers.

See also FRED-ROD's troubleshooting list (Section 15) for issues common to both apps (`IDACalcIC` failures, gap-closure setup, HDF5/`h5py` availability, Python module path issues).

25 Beyond FRED-ROD and FRED-OX: FRED-M-Na

FRED-M-Na (metallic U-Pu-Zr fuel in sodium) is built on the same platform layer as FRED-ROD and FRED-OX but is not covered in depth by this manual. Its examples live under `examples/fred_m_na/` and follow the same general pattern (geometry → materials → solver → run), with its own physics documented in `doc/physics_fred_m_na.md` and `doc/theory_manual/fred_m_na.tex`.

Example	What it shows
`simple_irradiation`	First FRED-M-Na run: single-pin base irradiation
`hot_start_demo`	Cold vs hot start (same concept as Sections 8, 18)
`restart_snapshot`	Checkpoint and snapshot persistence (same pattern as Section 21)
`timpano_SAS_benchmark`	Multi-year benchmark against legacy FRED-M / SAS4A

Where to start with FRED-M-Na

If you are already comfortable with FRED-ROD and FRED-OX from this manual, `simple_irradiation` → `hot_start_demo` → `restart_snapshot` follows the same learning progression as Parts I–II, before moving on to the full `timpano_SAS_benchmark` legacy-validation case.